BlackJack Advisor

Riconoscimento automatico di mani di blackjack e applicazione della strategia fondamentale tramite visione artificiale

ABSTRACT.

Blackjack Advisor è un sistema intelligente in grado di analizzare una mano di blackjack e suggerire automaticamente la mossa probabilisticamente ottimale, applicando la strategia fondamentale. Il sistema sfrutta tecniche di visione artificiale e apprendimento profondo, articolandosi in più fasi.

1.0 Introduzione

Il **Blackjack** è uno dei giochi di azzardo più diffusi al mondo, probabilmente per la sua semplicità e la sua probabilità di vittoria: applicando infatti una serie di regole basate sul calcolo delle probabilità, è possibile raggiungere una **percentuale di vittoria del 49%**, la più alta per un gioco d'azzardo contro il banco (naturalmente il banco continua a detenere il vantaggio probabilistico).

Nel gioco del BlackJack, vengono distribuite due carte scoperte al player e una al dealer: l'obiettivo è quello di chiedere carte aggiuntive o stare per ottenere il numero più alto possibile senza superare il 21. Il dealer giocherà poi allo stesso modo, con il vincolo di chiedere nuove carte finché il suo valore è al di sotto del 17. A questo punto, se il player avrà un punteggio maggiore del dealer, ma minore o uguale a 21, otterrà la vittoria di quella mano.

Possiamo vedere, quindi, come sia un gioco molto semplice e, attraverso l'utilizzo della strategia fondamentale, pubblicamente nota, è possibile divertirsi, e allo stesso tempo massimizzare le probabilità di vittoria, anche per giocatori non esperti.

Da questi presupposti, nasce **BlackJack Advisor**, il quale, dato in input uno screenshot o un'immagine della propria partita, applica in automatico la strategia fondamentale in maniera semplice ed immediata.

Il nostro progetto si sviluppa in quattro fasi principali:

1. **Object Detection** tramite YOLO
2. **Classificazione** tramite Convolutional Neural Network (CNN)
3. Applicazione della **strategia fondamentale** tramite algoritmo Python
4. Semplice **GUI** tramite Gradio, per rendere l'utilizzo user friendly

2.0 Fase 1 – Object Detection con YOLO

Per la prima fase del nostro progetto è stata utilizzata una delle architetture più popolari per l'object detection in tempo reale.

YOLO (You Only Look Ones) è un algoritmo di deep learning basato su CNN, il quale identifica e localizza oggetti all'interno di un'immagine, restituendo per ogni oggetto rilevato:

- Un **bounding box** (x1, y1, x2, y2)
- La **classe** (nel nostro caso unica "card")
- **Confidenza**, ovvero la probabilità che la predizione sia corretta.

2.1.1 YOLO Dataset

Per il train di YOLO abbiamo utilizzato un dataset misto formato da:

1. ***"Playing Cards Object Detection Dataset"*** , un dataset pubblico scaricato da **Kaggle** al seguente link https://www.kaggle.com/datasets/andy8744/playing-cards-object-detection-dataset.
   Data la grandezza del dataset, abbiamo deciso di rimpicciolirlo, tramite uno script python, per arrivare ad un totale di
   - 4.000 immagini per il train
   - 1.000 immagini per il validation
   - 1.000 immagini per il test

2. Un piccolo dataset creato ad hoc, utilizzando il software **Roboflow**. Abbiamo, quindi, eseguito 40 Screenshot di mani di blackjack, tramite un gioco online gratuito.
   Successivamente, attraverso Roboflow, abbiamo ritagliato manualmente i Bounding Box desiderati.
   Infine abbiamo eseguito una piccola **Augmentation** (Rotation, Saturation, Blur, Noise) per portare il numero delle immagini a 101.

2.1.2 YOLO Train

Una volta ottenuto il dataset desiderato, sono stati svolti **diversi cicli di train**, per ottenere i risultati migliori dalla rete.

1. Inizialmente è stato eseguito un train di 30 epoche, utilizzando solamente il dataset numero 1. Abbiamo notato come il modello così allenato funzionasse molto bene con immagini più "realistiche" e meno con immagini più artificiali (come quelle da noi utilizzate).

2. È stato così eseguito un nuovo allenamento di 20 epoche, utilizzando il dataset numero 2 e partendo dal modello già allenato in precedenza.

2.1.3 YOLO Results

Durante il training del modello **YOLOv8s**, sono stati ottenuti ottimi risultati sia in termini di ottimizzazione delle loss, sia di performance sulle metriche di rilevamento.

Le **componenti di loss** si sono ridotte significativamente nelle prime epoche, convergendo stabilmente intorno alla ventesima epoca. Nonostante alcune fluttuazioni nella validazione, il modello ha mostrato un comportamento robusto e **privo di overfitting significativo**.

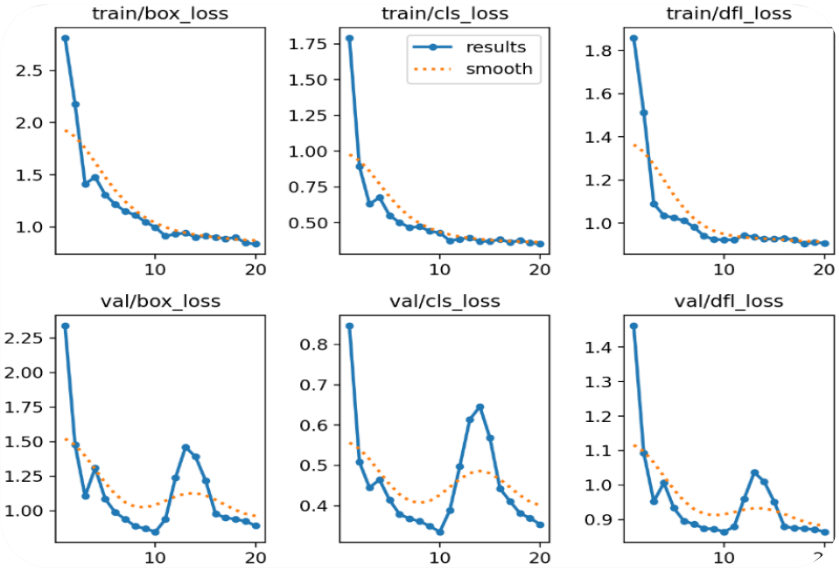


Figura 10 - Yolo Loss Results

Le **metriche di precisione e richiamo** si sono mantenute sopra lo 0.97 per quasi tutto il training, indicando una **eccellente capacità di localizzazione e classificazione dei bounding box delle carte**, anche con margini di accuratezza più stretti.

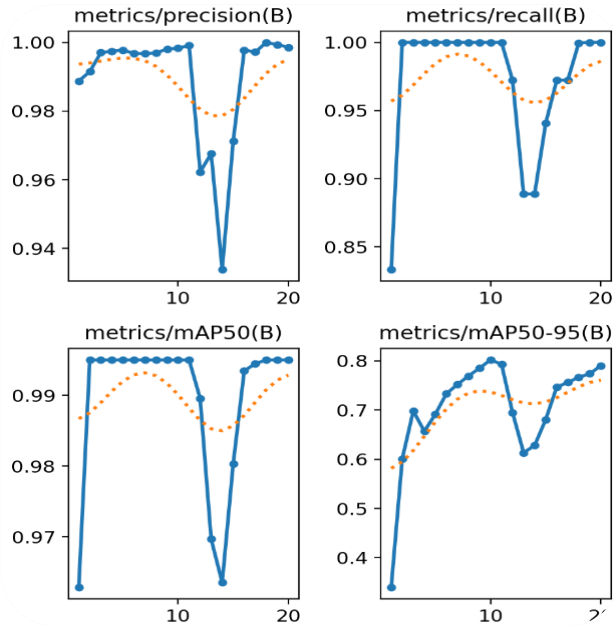


Figura 2 - YOLO Precision Metrics

Figura 3 - YOLO val batch pred

3.0 Fase 2 – Classificazione tramite CNN

L'obiettivo di questa seconda fase è classificare il numero e il seme di ogni carta individuata da YOLO.

È stata quindi costruita una **Convolutional Neural Nertwork (CNN)** personalizzata, la quale riceve in input il ritaglio dell'angolo superiore sinistro della carta e gestisce in parallelo due task di classificazione:

- Il riconoscimento del **seme** (4 classi)
- Il riconoscimento del **numero** (13 classi)

3.1 CNN Dataset

Anche per il Train della nostra CNN è stato utilizzato un dataset misto, formato da:

1. **Ritagli effettuati tramite uno script Python**, partendo da due dataset scaricati da Kaggle ai seguenti link:
   - https://www.kaggle.com/datasets/gpiosenka/cards-image-datasetclassification
   - https://www.kaggle.com/datasets/jamesmcguigan/playingcards

   Abbiamo generato quindi, a partire dai dataset di partenza contenenti complessivamente 51 immagini di carte intere per ogni tipologia di carta, un dataset formato da 3900 ritagli dell'angolo a sinistra superiore, applicando inoltre leggere augmentazioni di luminosità, sfocatura e nitidezza.

2. **Bounding box generati direttamente dalla rete di YOLO**. Per allenare la CNN anche su immagini più simili a quelle che poi avremmo effettivamente utilizzato, abbiamo deciso di includere nel dataset anche una serie di bounding box generati direttamente dalla rete di YOLO precedentemente allenata, per un totale di 164 immagini.

3.2 CNN Architecture

Come abbiamo visto, la nostra CNN è della tipologia nominata a **doppia testa**, in quanto progettata per effettuare due classificazioni indipendenti a partire dalla stessa immagine di input.

Dopo numerose modifiche, basate su prove di allenamento della rete, la struttura finale è composta come segue:

1. Un **Feature extractor condiviso**, costituito da quattro blocchi convoluzionali $Conv2d \rightarrow ReLu \rightarrow MaxPool$ seguiti da un layer AdaptiveAvgPool2d, il quale forza l'output a dimensione fissa (256, 2, 2)

2. A seguito di un'operazione di Flatten, le feature vengono elaborate da due rami separati:
   a. **Una testa per il seme**, con tre layer fully connected progressivamente più piccoli e dropout per ridurre l'overfitting;
   b. **Una testa per il numero**, simile alla precedente.

3. Entrambe le teste terminano con un layer Linear che restituisce i logit per ciascuna classe.

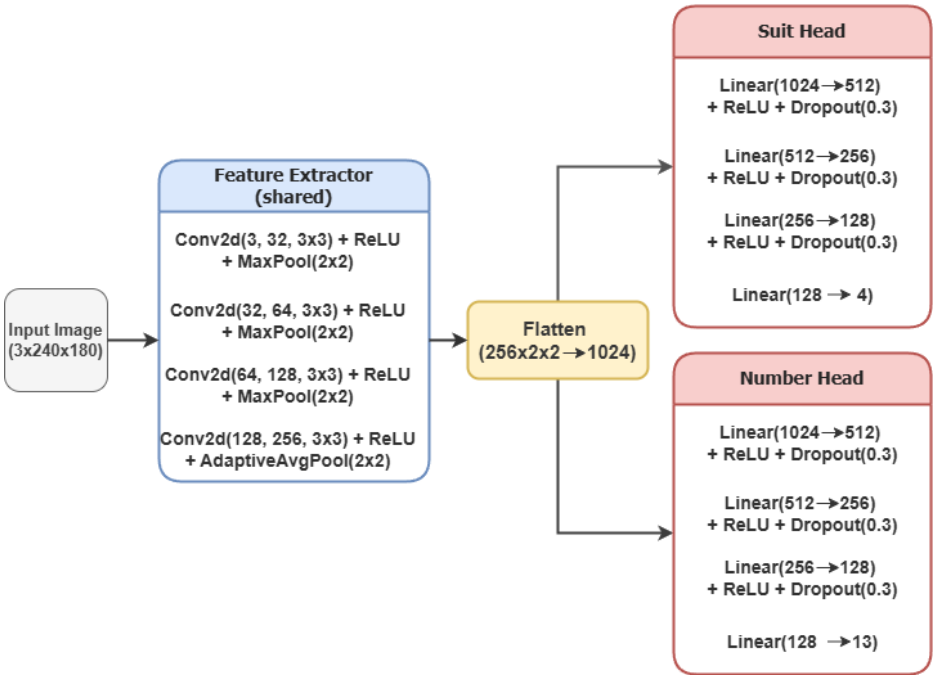


Figura 4 - CNN Architecture

3.3 CNN Train and Results

Abbiamo effettuato numerosi train, sostituendo gli ipermarametri e modificando l'architettura della CNN per ottenere risultati migliori, basati sui risultati ottenuti.

Il train finale è stato eseguito sull'architettura mostrata nel paragrafo precedente, utilizzando i seguenti iperparametri:

- **Epochs**: 30
- **Batch-size**: 32
- **Learning rate**: 0.001
- **Optimizer**: Adam

Le performance sono state monitorate separatamente per ciascun task, valutando **loss** e **accuracy** su training e validation set.

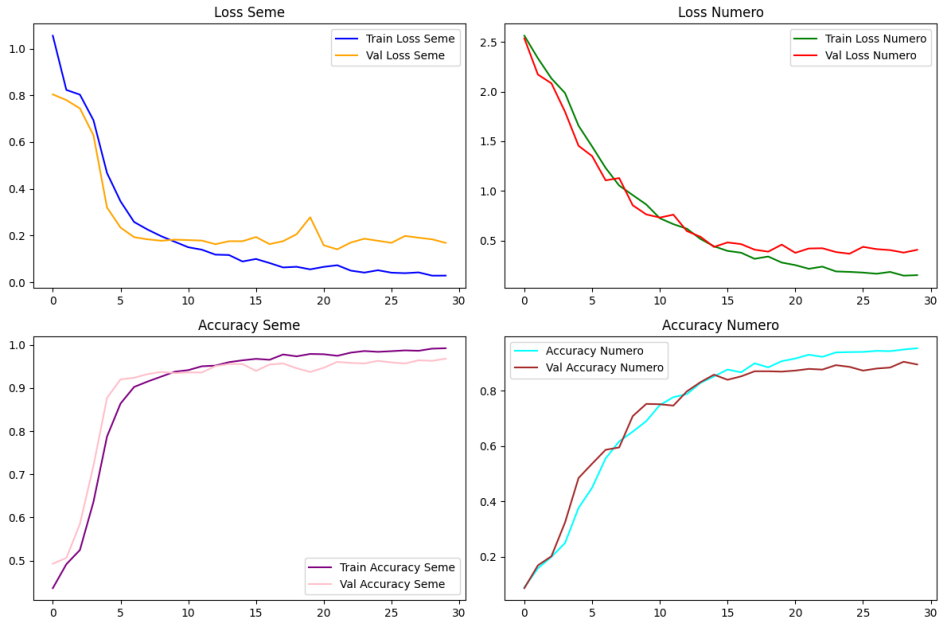


Figura 5 - CNN Train Results

Il modello ha raggiunto **ottime performance su entrambi i compiti**, con una **eccellente accuratezza sul seme** e una **buona capacità di generalizzazione anche sul valore numerico**, nonostante quest'ultimo risulti leggermente più difficile da classificare.

- Per il **seme**, è stato raggiunto il ~99% di accuracy durante il tranining con una loss function prossima allo zero, mentre durante la validation è stato raggiunto il ~97% di accuracy con una loss function di 0.2.

- Per il **numero**, è stato raggiunto il ~97% di accuracy durante il training con una loss function ~0.2, mentre durante la validation è stato raggiunto i ~90% di accuracy con una loss function di 0.5.

Questi risultati, sono confermati anche dalle confusion matrix risultanti.

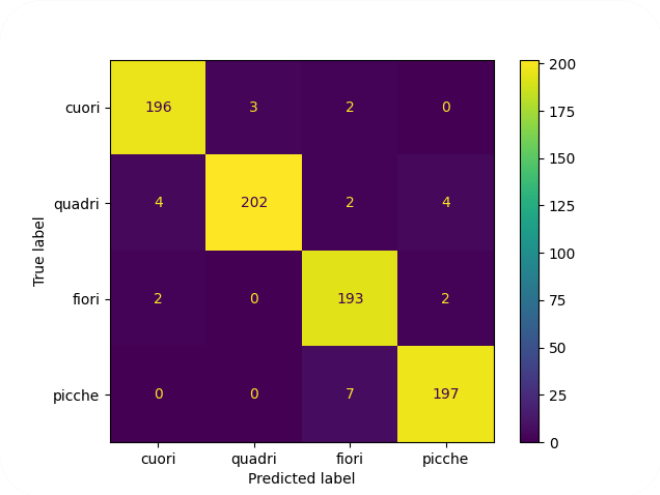


Figura 6a - Suit Confusion Matrix

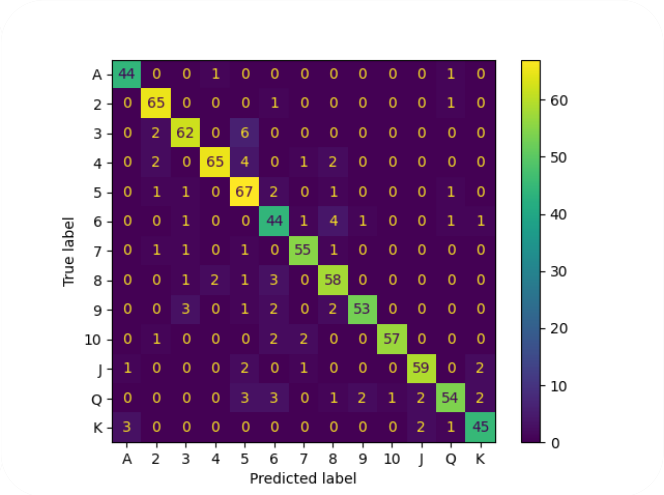


Figura 6b - Number Confusion Matrix

4.0 Fase 3 – Strategia Fondamentale

Questa fase si occupa di prendere in input le etichette generate dalla CNN ed applicare la **strategia fondamentale** introdotta in precedenza, per restituire al player la **mossa probabilisticamente ideale** nella situazione specifica.

GIOCATORE	CARTA SCOPERTA DEL BANCO									
	2	3	4	5	6	7	8	9	10	A
8	CARTA	CARTA	CARTA	CARTA	CARTA	CARTA	CARTA	CARTA	CARTA	CARTA
9	CARTA	DOPPIO	DOPPIO	DOPPIO	DOPPIO	CARTA	CARTA	CARTA	CARTA	CARTA
10	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	CARTA	CARTA
11	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	CARTA
12	CARTA	CARTA	STAI	STAI	STAI	CARTA	CARTA	CARTA	CARTA	CARTA
13	STAI	STAI	STAI	STAI	STAI	CARTA	CARTA	CARTA	CARTA	CARTA
14	STAI	STAI	STAI	STAI	STAI	CARTA	CARTA	CARTA	CARTA	CARTA
15	STAI	STAI	STAI	STAI	STAI	CARTA	CARTA	CARTA	CARTA	CARTA
16	STAI	STAI	STAI	STAI	STAI	CARTA	CARTA	CARTA	CARTA	CARTA
17	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI
A - 9	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI
A - 8	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI
A – 7	STAI	DOPPIO	DOPPIO	DOPPIO	DOPPIO	STAI	STAI	CARTA	CARTA	CARTA
A - 6	CARTA	DOPPIO	DOPPIO	DOPPIO	DOPPIO	CARTA	CARTA	CARTA	CARTA	CARTA
A - 5	CARTA	CARTA	DOPPIO	DOPPIO	DOPPIO	CARTA	CARTA	CARTA	CARTA	CARTA
A – 4	CARTA	CARTA	DOPPIO	DOPPIO	DOPPIO	CARTA	CARTA	CARTA	CARTA	CARTA
A – 3	CARTA	CARTA	CARTA	DOPPIO	DOPPIO	CARTA	CARTA	CARTA	CARTA	CARTA
A – 2	CARTA	CARTA	CARTA	DOPPIO	DOPPIO	CARTA	CARTA	CARTA	CARTA	CARTA
A - A	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT
10 - 10	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI	STAI
9 - 9	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	STAI	SPLIT	SPLIT	STAI	STAI
8 - 8	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT
7 -7	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	CARTA	CARTA	CARTA	CARTA
6 - 6	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	CARTA	CARTA	CARTA	CARTA	CARTA
5 - 5	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	DOPPIO	CARTA	CARTA
4 - 4	CARTA	CARTA	CARTA	SPLIT	SPLIT	CARTA	CARTA	CARTA	CARTA	CARTA
3 - 3	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	CARTA	CARTA	CARTA	CARTA
2 - 2	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	SPLIT	CARTA	CARTA	CARTA	CARTA

È stato quindi generato un piccolo Script Python, il quale riceve in input un array contenente le carte del player e la carta del dealer e restituisce in output una stringa in [“HIT”, “STAND”, “DOUBLE”, “SPLIT”].

5.0 Fase 4 – GUI

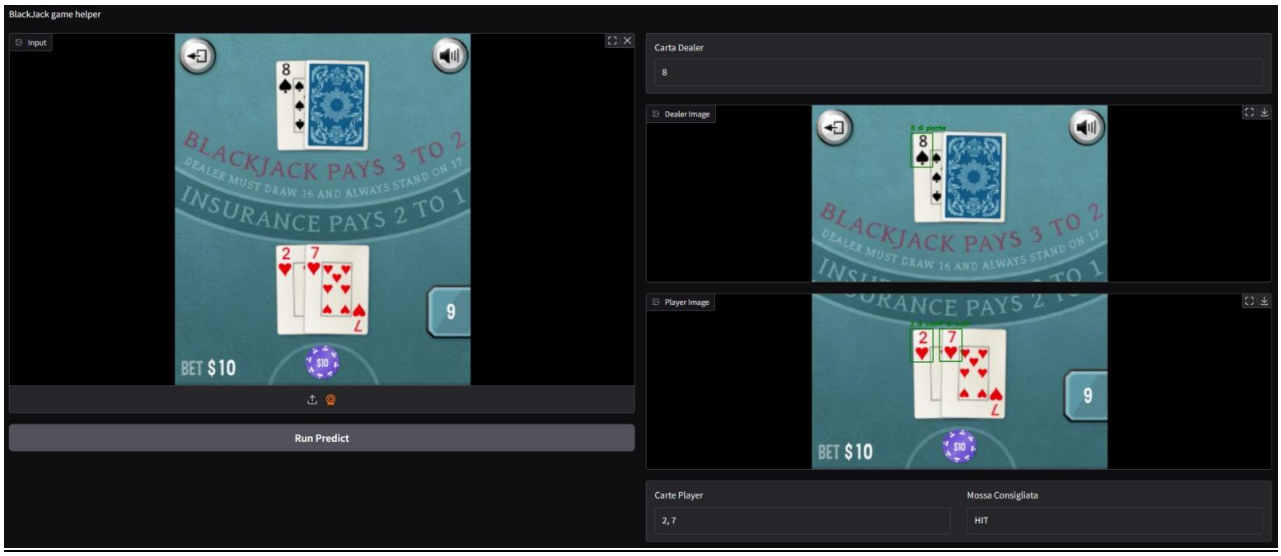
In questa ultima fase è stata svolta un’operazione di integrazione delle diverse componenti del progetto.

È stato, quindi, implementato uno script principale, denominato main.py, il quale coordina l’ interazione tra le varie sezioni del sistema.

Per rendere l’utilizzo del software più intuitivo e accessibile all’utente finale, è stata inoltre sviluppata una **semplice interfaccia grafica (GUI)** utilizzando la libreria **Gradio**, che consente di costruire interfacce interattive in modo rapido ed efficace.

L’utente finale avrà quindi la possibilità di caricare un’immagine della propria partita per ricevere in output la stessa immagine opportunatamente processata mostrando:

- **Bounding box** generati dal modello YOLO;
- **Etichette** generate dalla CNN;
- **Suggerimento** sulla mossa migliore da fare in termini di probabilità.



6.0 Sviluppi Futuri

Gli **obiettivi** di questo progetto per il **futuro** sono:

- Aggiungere la possibilità di utilizzo del software in tempo reale, utilizzando uno stream video come input.
- Aggiornare l’algoritmo, per implementare un sistema di conteggio delle carte, il quale aumenterebbe significativamente la probabilità di vittoria del player.

Partecipanti

- Giovanni Ciarravano, 1989209
- Federico Iannucci, 2060656
- Marco Linardi, 2003980
- Emanuele Bruni, 2067656